

Apache Cassandra Implementation for Music Streaming Data Management

Objective

The objective of this project is to install and configure Apache Cassandra on Ubuntu WSL, create a NoSQL database for a music streaming application, and design optimized tables to efficiently answer application-specific queries. The project demonstrates Cassandra data modeling, table creation, data insertion, and query execution using CQL.

Project Description

This project uses **Apache Cassandra**, a distributed NoSQL database, to build a database for the **Apache Cassandra Implementation for Music Streaming Data Management**.

Initially, Cassandra is installed using Docker (recommended for WSL) or through a direct Ubuntu installation. A keyspace named **sparkify** is created, followed by three tables designed specifically to support different query patterns:

- Retrieve all songs played in a specific session.
- Retrieve all songs played by a user during a session in reverse chronological order.
- Find all users who listened to a particular song.

Sample data is inserted into each table, and CQL queries are executed to verify the database design and retrieve the required information. The project also includes optional Python-based data generation for testing with larger datasets and demonstrates Cassandra features such as UPSERT operations, partition keys, clustering order, and consistency levels.

Screenshots

```
sky@MS97377: ~
sky@MS97377:~$
sky@MS97377:~$ # Test connection
sky@MS97377:~$ docker exec cassandra cqlsh -e "DESC KEYSPACES"

system      system_distributed  system_traces  system_virtual_schema
system_auth  system_schema      system_views

sky@MS97377:~$ # Run the CQL setup
sky@MS97377:~$ docker exec -i cassandra cqlsh -f /scripts/setup_exercise.cql

sparkify     system_auth          system_schema  system_views
system       system_distributed   system_traces  system_virtual_schema

song_info_by_session  song_playing_history_by_user  who_listened_to_song

session_id | item_in_session | artist      | length | song
-----
100        | 1               | The Beatles | 4.32   | Hey Jude
100        | 2               | Pink Floyd | 6.48   | Comfortably Numb
100        | 3               | Queen      | 5.55   | Bohemian Rhapsody
101        | 1               | Led Zeppelin | 8.02   | Stairway to Heaven
101        | 2               | Nirvana     | 5.01   | Smells Like Teen Spirit
(5 rows)

user_id | session_id | item_in_session | artist      | first_name | last_name | song
-----
1       | 101        | 1               | Led Zeppelin | John       | Doe       | Stairway to Heaven
1       | 100        | 1               | The Beatles | John       | Doe       | Hey Jude
1       | 100        | 2               | Pink Floyd | John       | Doe       | Comfortably Numb
2       | 100        | 1               | The Beatles | Jane       | Smith     | Hey Jude
(4 rows)

song      | user_id | first_name | last_name
-----
Stairway to Heaven | 1       | John       | Doe
Hey Jude          | 1       | John       | Doe
Hey Jude          | 2       | Jane       | Smith
(3 rows)
sky@MS97377:~$
```


session_id	item_in_session	artist	length	song
100	1	The Beatles	4.03	Let It Be
100	2	Pink Floyd	6.48	Comfortably Numb
100	3	Queen	5.55	Bohemian Rhapsody

```
cqlsh:sparkify>
```

```
cqlsh:sparkify> -- View all data
cqlsh:sparkify> SELECT * FROM song info by session;
```

session_id	item_in_session	artist	length	song
100	1	The Beatles	4.03	Let It Be
100	2	Pink Floyd	6.48	Comfortably Numb
100	3	Queen	5.55	Bohemian Rhapsody
102	1	Nirvana	5.01	Smells Like Teen Spirit
101	1	Led Zeppelin	8.02	Stairway to Heaven
101	2	Nirvana	5.01	Smells Like Teen Spirit

```
cqlsh:sparkify> SELECT * FROM song_playing_history_by_user;
```

user_id	session_id	item_in_session	artist	first_name	last_name	song
1	101	1	Led Zeppelin	John	Doe	Stairway to Heaven
1	100	1	The Beatles	John	Doe	Hey Jude
1	100	2	Pink Floyd	John	Doe	Comfortably Numb
2	100	1	The Beatles	Jane	Smith	Hey Jude

```
cqlsh:sparkify> SELECT * FROM who_listened_to_song;
```

session_id	item_in_session	artist	length	song
100	1	The Beatles	4.03	Let It Be
100	2	Pink Floyd	6.48	Comfortably Numb
100	3	Queen	5.55	Bohemian Rhapsody
102	1	Nirvana	5.01	Smells Like Teen Spirit
101	1	Led Zeppelin	8.02	Stairway to Heaven
101	2	Nirvana	5.01	Smells Like Teen Spirit

```
cqlsh:sparkify> SELECT * FROM song_playing history_by_user;
```

user_id	session_id	item_in_session	artist	first_name	last_name	song
1	101	1	Led Zeppelin	John	Doe	Stairway to Heaven
1	100	1	The Beatles	John	Doe	Hey Jude
1	100	2	Pink Floyd	John	Doe	Comfortably Numb
2	100	1	The Beatles	Jane	Smith	Hey Jude

```
cqlsh:sparkify> SELECT * FROM who_listened_to_song;
```

song	user_id	first_name	last_name
Stairway to Heaven	1	John	Doe
Hey Jude	1	John	Doe
Hey Jude	2	Jane	Smith

```
cqlsh:sparkify>
```

```
cqlsh:sparkify> -- Exit
```

```
cqlsh:sparkify> EXIT;
```

```
sky@MS97377:~$ docker exec cassandra cqlsh -e "USE sparkify; SELECT * FROM song info by session;"
```

```
Check table structure
docker exec cassandra cqlsh -e 'USE sparkify; SELECT * FROM song
```

```
docker exec cassandra cqlsh -e "USE sparkify; DESC TABLE song_info_by_session;"
session id | item in session | artist | length | song
```

session_id	item_in_session	artist	length	song
100	1	The Beatles	4.03	Let It Be
100	2	Pink Floyd	6.48	Comfortably Num
100	3	Queen	5.55	Bohemian Rhapsody
102	1	Nirvana	5.01	Smells Like Teen Spirit
101	1	Led Zeppelin	8.02	Stairway to Heaven
101	2	Nirvana	5.01	Smells Like Teen Spirit

sky@MS97377:~\$

```
sky@MS97377:~$ # Check table structure
```

```
sky@MS97377:~$ docker exec cassandra cqlsh -e "USE sparkify; DESC TABLE song_info_by_session;"
```

```
CREATE TABLE sparkify.song_info_by_session (
  session_id int,
  item_in_session int,
  artist text,
  length float,
  song text,
  PRIMARY KEY (session_id, item_in_session)
) WITH CLUSTERING ORDER BY (item_in_session ASC)
AND additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.01
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';
```

sky@MS97377:~\$

```
sky@MS97377: ~/cassandra-exercise
sky@MS97377:~$ docker info
Client: Docker Engine - Community
Version: 29.6.1
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.35.0
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v5.3.1
    Path: /usr/libexec/docker/cli-plugins/docker-compose
Server:
  Containers: 1
    Running: 0
    Paused: 0
    Stopped: 1
  Images: 1
  Server Version: 29.6.1
  Storage Driver: overlayfs
    driver-type: io.containerd.snapshotter.v1
  Logging Driver: json-file
  Cgroup Driver: systemd
  Cgroup Version: 2
  Plugins:
    Volume: local
    Network: bridge host ipvlan macvlan null overlay
    Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
  CDI spec directories:
    /etc/cdi
    /var/run/cdi
  Swarm: inactive
  Runtimes: runc io.containerd.runc.v2
  Default Runtime: runc
  Init Binary: docker-init
  containerd version: e53c7c1516c3b2b98eb76f1f4117477e6f4e66
  runc version: v1.3.6-0-g491b69ba
  init version: de40ad0
  Security Options:
    seccomp
    Profile: builtin
    cgroupns
  Kernel Version: 6.18.33.2-microsoft-standard-WSL2

sky@MS97377: ~/cassandra-exercise
cgroupns
Kernel Version: 6.18.33.2-microsoft-standard-WSL2
Operating System: Ubuntu 22.04.5 LTS
OSType: linux
Architecture: x86_64
CPUs: 4
Total Memory: 3.766GiB
Name: MS97377
ID: 542ad5ee-7d98-4334-b95d-94c09ed4022a
Docker Root Dir: /var/lib/docker
Debug Mode: false
Experimental: false
Insecure Registries:
  ::1/128
  127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
  EnableUserlandProxy: true
  UserlandProxyPath: /usr/bin/docker-proxy

sky@MS97377:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
sky@MS97377:~$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
e4f402f8e66a   hello-world  "/hello"                4 minutes ago  Exited (0) 4 minutes ago  competent_curran
sky@MS97377:~$ docker run -it ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
2c1ce1d0a589: Pull complete
a9be9fd915e9: Pull complete
bce5ee84b2b4: Download complete
Digest: sha256:b7f48194d4d8b763a478a621cdc81c27be222ba2206ca3ca6bc42b49685f3d9e
Status: Downloaded newer image for ubuntu:latest
root@4250a7eb2e07:/# docker run -d -p 8080:80 nginx
bash: docker: command not found
root@4250a7eb2e07:/#
root@4250a7eb2e07:/# cd
root@4250a7eb2e07:/# exit
exit
sky@MS97377:~$ mkdir -p ~/cassandra-exercise
sky@MS97377:~$ cd ~/cassandra-exercise
sky@MS97377:~/cassandra-exercise$ cat > docker-compose.yml << 'EOF'
services:
> cassandra:
```

```
sky@MS97377: ~/cassandra-exercise
root@4250a7eb2e07:~# exit
exit
sky@MS97377:~$ mkdir -p ~/cassandra-exercise
sky@MS97377:~$ cd ~/cassandra-exercise
sky@MS97377:~/cassandra-exercise$ cat > docker-compose.yml << 'EOF'
services:
  cassandra:
    image: cassandra:latest
    container_name: cassandra
    hostname: cassandra
    ports:
      - "9042:9042"
      - "9160:9160"
    volumes:
      - "/cassandra-data:/var/lib/cassandra"
scripts:
  - "/scripts/scripts"
environment:
  - CASSANDRA_CLUSTER_NAME=MyCluster
  - CASSANDRA_DC=dc1
  - CASSANDRA_RACK=rack1
  - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
  - CASSANDRA_NUM_TOKENS=256
  - MAX_HEAP_SIZE=512M
  - HEAP_NEWSIZE=100M
  restart: unless-stopped
EOF
t Cassandra
docker compose up -d
# Wait for Cassandra to initialize (60 seconds)
echo "sky@MS97377:~/cassandra-exercise$ # Start Cassandra
sky@MS97377:~/cassandra-exercise$ docker compose up -d
Waiting for Cassandra to start (60 seconds)..."
sleep 60
# Check if container is running
docker ps | grep cassandra
services must be a mapping
sky@MS97377:~/cassandra-exercise$ # Wait for Cassandra to initialize (60 seconds)
sky@MS97377:~/cassandra-exercise$ echo "Waiting for Cassandra to start (60 seconds)..."
Waiting for Cassandra to start (60 seconds)..."
sky@MS97377:~/cassandra-exercise$ sleep 60
sky@MS97377:~/cassandra-exercise$ # Check if container is running
sky@MS97377:~/cassandra-exercise$ docker ps | grep cassandra
sky@MS97377:~/cassandra-exercise$
```

```
sky@MS97377:~/cassandra-exercise
sky@MS97377:~/cassandra-exercise$ echo "Waiting for Cassandra to start (60 seconds)..."
Waiting for Cassandra to start (60 seconds)..."
sky@MS97377:~/cassandra-exercise$ sleep 60
sky@MS97377:~/cassandra-exercise$ # Check if container is running
sky@MS97377:~/cassandra-exercise$ docker ps | grep cassandra
sky@MS97377:~/cassandra-exercise$ docker compose up -d
services must be a mapping
sky@MS97377:~/cassandra-exercise$ echo "Waiting for Cassandra to start (60 seconds)..."
Waiting for Cassandra to start (60 seconds)..."
sky@MS97377:~/cassandra-exercise$ sleep 60
sky@MS97377:~/cassandra-exercise$ docker ps | grep cassandra
sky@MS97377:~/cassandra-exercise$ docker logs cassandra | tail -20
Error response from daemon: No such container: cassandra
sky@MS97377:~/cassandra-exercise$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
4250a7eb2e07   ubuntu    "bash"                  17 minutes ago   Exited (0)    14 minutes ago             happy_wing
e4f402f8e66a   hello-world "/hello"                22 minutes ago   Exited (0)    22 minutes ago             competent_curran
sky@MS97377:~/cassandra-exercise$ sudo docker run -d \
--name cassandra \
-p 9042:9042 \
cassandra:latest
[sudo] password for sky:
Unable to find image 'cassandra:latest' locally
latest: Pulling from library/cassandra
2bce2595dd56: Pull complete
8773db3c24c9: Pull complete
916a6e2b19fa: Pull complete
78445e2fd27e: Pull complete
68629629b516: Pull complete
d99d06a4e22d: Pull complete
04520a8b8dad: Pull complete
3c55eeaca078: Pull complete
37bd7b8fc716: Download complete
faa2e82bdefb: Download complete
Digest: sha256:b89056c366c4b807380cd9a4ac865ad558c0b4d5aac342f69087d0169ca1ddcd
Status: Downloaded newer image for cassandra:latest
a5ee3f41aa1b705aca913e24fa86b92ca52a37e19b3acd979ee5f90138fe951
sky@MS97377:~/cassandra-exercise$ sudo docker ps -a
CONTAINER ID   IMAGE      NAMES      COMMAND                  CREATED        STATUS        PORTS
a5ee3f41aa1b   cassandra:latest   "docker-entrypoint.s..."  30 seconds ago   Up 25 seconds    7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::]:
9042->9042/tcp   cassandra
4250a7eb2e07   ubuntu    happy_wing   "bash"                  19 minutes ago   Exited (0)    17 minutes ago
```

```

sky@MS97377: ~/cassandra-exercise
aSee3f41aa1b705aca913e24fa86b92ca52a37e19b3acdd979ee5f90138fe951
sky@MS97377:~/cassandra-exercise$ sudo docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
aSee3f41aa1b   cassandra:latest   "docker-entrypoint.s..." 30 seconds ago Up 25 seconds   7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [
9042->9042/tcp   cassandra
4250a7eb2e07   ubuntu      "bash"                  19 minutes ago Exited (0) 17 minutes ago
e4f402fbc66a   hello-world    "/hello"                 25 minutes ago Exited (0) 25 minutes ago
competent_curran

sky@MS97377:~/cassandra-exercise$ sudo docker logs cassandra | tail -20
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by org.apache.cassandra.security.ThreadAwareSecurityManager (file:/opt/cassandra/lib/apache-cassandra-5.0.8.jar)
WARNING: Please consider reporting this to the maintainers of org.apache.cassandra.security.ThreadAwareSecurityManager
WARNING: System::setSecurityManager will be removed in a future release
INFO [main] 2026-07-09T03:30:59,045 LocalSessions.java:416 - LocalSessions start completed in 4 ms, sessions loaded from DB: 0
INFO [main] 2026-07-09T03:30:59,048 StreamManager.java:267 - Storing streaming state for 3d or for size 41943040
INFO [main] 2026-07-09T03:30:59,051 QueryProcessor.java:198 - Preloaded 0 prepared statements in 1 ms
INFO [main] 2026-07-09T03:30:59,052 StorageService.java:968 - Cassandra version: 5.0.8
INFO [main] 2026-07-09T03:30:59,052 StorageService.java:969 - Git SHA: 172227094626d50d4bc9a3d9de2e3659a7844386
INFO [main] 2026-07-09T03:30:59,052 StorageService.java:970 - CQL version: 3.4.7
INFO [main] 2026-07-09T03:30:59,053 StorageService.java:971 - Native protocol supported versions: 3/v3, 4/v4, 5/v5, 6/v6-beta (default: 5/v5)
INFO [main] 2026-07-09T03:30:59,094 IndexSummaryManager.java:105 - Initializing index summary manager with a memory pool size of 96 MB and a resize interval of 60 m
tes
INFO [main] 2026-07-09T03:30:59,095 StorageService.java:988 - Loading persisted ring state
INFO [main] 2026-07-09T03:30:59,096 StorageService.java:1086 - Populating token metadata from system tables
INFO [main] 2026-07-09T03:30:59,118 InboundConnectionInitiator.java:164 - Listening on address: (/172.17.0.2:7000), nic: eth0, encryption: unencrypted
INFO [main] 2026-07-09T03:30:59,304 StorageService.java:880 - Unable to gossip with any peers but continuing anyway since node is in its own seed list
INFO [main] 2026-07-09T03:30:59,480 StorageService.java:1216 - Starting up server gossip
INFO [main] 2026-07-09T03:30:59,489 ColumnFamilyStore.java:1052 - Enqueuing flush of system.local, Reason: INTERNALLY_FORCED, Usage: 525B (0%) on-heap, 0B (0%) off-
p
INFO [PerDiskMemtableFlushWriter_0:1] 2026-07-09T03:30:59,514 Flushing.java:153 - Writing Memtable-local@629937921(53B serialized bytes, 1 ops, 525B (0%) on-heap, 0
0%) off-heap), flushed range = [null, null)
INFO [PerDiskMemtableFlushWriter_0:1] 2026-07-09T03:30:59,516 Flushing.java:179 - Completed flushing /var/lib/cassandra/data/system/local-7ad54392bcdd35a684174e0478
377/nb-5-big-Data.db (29B) for commitlog position CommitLogPosition(segmentId=1783567854689, position=879)
INFO [MemtableFlushWriter:2] 2026-07-09T03:30:59,550 LogTransaction.java:249 - Unfinished transaction log, deleting /var/lib/cassandra/data/system/local-7ad54392bcd
a684174e047860b377/nb_txn_flush_9a7b8730-7b46-11f1-8aa2-679f1630c3bb.log
INFO [main] 2026-07-09T03:30:59,799 MigrationCoordinator.java:260 - Starting migration coordinator and scheduling pulling schema versions every PT1M
INFO [main] 2026-07-09T03:30:59,818 SnapshotManager.java:135 - Scheduling expired snapshot cleanup with initialDelaySeconds=5 and cleanupPeriodSeconds=60
INFO [main] 2026-07-09T03:31:00,000 StorageService.java:1294 - This node will not auto bootstrap because it is configured to be a seed node.
sky@MS97377:~/cassandra-exercise$

```

Learning Question:

1. Data Modeling: Why do we have 3 separate tables instead of 1 table with JOINS? Cassandra is a **NoSQL database** and does **not support JOIN operations** like relational databases. Therefore, data is modeled based on the application's query requirements. Three separate tables are created because each table is optimized for a specific query:

- **Table1:** Retrieve all songs played in a session.
- **Table2:** Retrieve all songs played by a specific user during a session.
- **Table3:** Find all users who listened to a particular song.

This approach provides faster query performance and avoids expensive joins.

2. PartitionKeys: What would happen if we used `user_id` alone as the partition key for Query 2?

If `user_id` alone were used as the partition key, then all sessions of the same user would be stored in the same partition. When retrieving songs for a specific session, Cassandra would have to scan through all of that user's session data, making the query slower and less efficient.

Using `(user_id, session_id)` as the partition key stores each user's session separately, allowing Cassandra to retrieve only the required session quickly.

3. Clustering Order: Why use ASC for one table and DESC for another? The clustering order is chosen based on how the data is queried:

- **ASC (Ascending)** is used in `song_info_by_sessions` so that songs appear in the order they were played during the session.
- **DESC (Descending)** is used in `song_playing_history_by_users` so that the most recently played songs appear first.

This matches the expected output of each query and improves retrieval efficiency.

4. UPSERT Behavior: What happens when you INSERT a row with an existing PRIMARY KEY?

In Cassandra, an **INSERT** operation with an existing **PRIMARY KEY** performs an **UPSERT**. This means the existing row is updated (overwritten) with the new values instead of creating a duplicate row.

For example, inserting a new song with the same `(session_id, item_in_session)` replaces the previous song stored at that primary key.

5. Consistency: What is the difference between CONSISTENCY ONE and CONSISTENCY ALL?

CONSISTENCY ONE	CONSISTENCY ALL
Requires a response from one replica only.	Requires responses from all replicas .
Faster read/write operations.	Slower because every replica must respond.
Higher availability.	Lower availability if any replica is unavailable.
May return slightly older data if replicas are not synchronized.	Ensures the latest data is returned because all replicas must agree.

Conclusion

This project demonstrates the practical implementation of Apache Cassandra for managing large-scale, distributed NoSQL databases. By designing query-specific tables instead of relying on joins, the Sparkify music streaming database efficiently retrieves session-based and user-based music information. The project also provides hands-on experience with Cassandra installation, keyspace creation, table design, CQL commands, data insertion, and query execution. Overall, the exercise highlights the importance of data modeling in Cassandra and shows how NoSQL databases provide scalable and high-performance solutions for big data applications.